Deadline-Aware Serving of LLM Agent Workloads: Batching, KV Caching, and Asynchronous Tool Calls

[Author names] [1]

Abstract

LLM agents interleave model calls with tool calls of unpredictable latency, and many agents, sometimes collaborating on one task, share one GPU. Each serving decision then affects every task's deadline: which requests to batch, which KV caches to keep, evict or preload, and whether a tool call blocks the agent or runs asynchronously. We formulate this as one online scheduling problem. We prove it NP-hard even in a trivial special case, so we target a constant-factor online algorithm, and we prove the first two reductions toward it: continuous batching is equivalent, up to a factor of 2 in speed, to a set of identical parallel machines, and issuing tool calls eagerly never hurts the optimum. All proofs are machine-checked (Z3 for the algebra, exhaustive and randomized tests for the combinatorics). On the systems side we built a deadline-enforcing agent runtime on vLLM and measured its first slice on an A40: asynchronous tool calls with KV *preloading*, which prefills tool results into the prefix cache while other tools are still running. On 20 held-out multi-hop questions with 8 parallel searches, preloading finished every task earlier (1.6 s on average, 95% CI 1.49–1.72) with identical answers, and cut deadline misses at 16 s from 45% to 15%.

1. Problem

Agents issue tool calls (search, code execution, APIs, other agents) whose latency is long and random compared with a model step, often several in parallel (Kim et al., 2024). Deployments put many agents on one GPU (setting a), and one task may itself be several collaborating agents, such as an orchestrator with sub-agents (setting b). Under end-to-end deadlines the serving layer faces coupled decisions: **batching** (running requests together raises throughput but slows each one), **KV memory** (keep a paused agent's cache through a tool wait, evict it and recompute later, or prefill tool results before they are needed), and **tool-call execution** (block the agent on a call, or continue and consume the result later).

Why it is new. Existing systems address pieces: vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024) batch requests and reuse prefixes, InferCept (Abhyankar et al., 2024) manages KV caches of requests paused for external calls, and Parrot (Lin et al., 2024) schedules multi-call LLM applications. None decides batching, KV and tool execution jointly against task deadlines, and none gives guarantees. Our goal is an online algorithm with a provable constant-factor guarantee, implemented as a layer on vLLM.

Challenges. Tool durations and output lengths are unknown when decisions are made; GPU time and KV memory are coupled (eviction saves memory but costs recompute); and deadlines belong to tasks, not to the individual model calls the engine sees.

2. Model and theory

Model. One GPU runs an iteration-level batching engine. A decode step with B running sequences takes $\tau(B) = a + cB$ seconds; KV memory holds M tokens. Task j arrives at r_j with deadline d_j and value w_j, and is a DAG of LLM nodes (model calls by any of its agents) and tool nodes (off-GPU calls with random durations). A tool result may be consumed at any later LLM node of its agent that the program allows (synchronous = the next node). Online, without knowing future arrivals, durations or output lengths, the runtime chooses batches, KV keep/evict/preload actions, and tool issue and consumption points, to maximize the total value of tasks finished by their deadlines. The full model is in the project documentation.

Theorem 1 (NP-hardness)**.** *The offline problem is NP-hard even with batch size 1, no tools, unlimited KV memory, deterministic lengths, single-node tasks and* $r_j = 0$.

Proof sketch. These restrictions give $1 \parallel \sum w_j U_j$. With a common deadline equal to a knapsack capacity, p_j = item

[1]Harvard University, CS 243 (Fall 2026). Midterm report. Correspondence to: [Author] <[email]>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

weight and w_j = item value, a job set is on time iff its total length fits, so the problem is Knapsack (Karp, 1972). □

With release dates and no preemption even unit values are strongly NP-hard (Lenstra et al., 1977), and online, no deterministic algorithm on one machine guarantees more than $1/4$ of the optimum even when value is proportional to work (Baruah et al., 1992); with unbounded value densities no constant ratio is possible (Koren & Shasha, 1995). Since the problem must be solved online anyway, **we target a constant-factor competitive algorithm under resource augmentation** (Kalyanasundaram & Pruhs, 2000): the algorithm may use a constant-factor faster GPU or larger KV memory than the optimum it is compared against. We reduce each resource to a classical online problem.

Lemma 2 (Batching → parallel machines)**.** *If $m = a/c$ is an integer, (i) any schedule on m identical machines of speed $1/(2a)$ (preemption, migration) can be executed by the engine with no completion later, and (ii) any engine schedule can be executed on m machines of speed $1/a$.*

Proof. (i) At most m jobs run, so $B \le a/c$ and each sequence advances at $1/(a+cB) \ge 1/(2a)$. (ii) Each sequence advances at $\le 1/a$ and the batch at $B/(a+cB) < 1/c = m/a$; such a rate vector is realizable on m machines of speed $1/a$ by McNaughton's wrap-around rule (McNaughton, 1959). □

Hence a C-competitive online algorithm for identical machines with speed augmentation σ becomes C-competitive on the engine with augmentation 2σ (a further factor ≤ 2 if a/c is not an integer and $a \ge 2c$).

Lemma 3 (Eager asynchronous tool calls)**.** *If tools share no capacity limit and their durations and results do not depend on issue or consumption time, issuing every tool call as soon as its inputs exist never lowers the offline optimum.*

Proof. Earlier issue only makes dependent nodes available earlier; every feasible schedule stays feasible. □

With eager issue, each task becomes a stream of LLM-node jobs released online, the setting of the remaining steps: (1) per-node sub-deadlines from task deadlines and remaining critical paths, with deadline-ordered, value-density-based admission on the machines of Lemma 2 (Koren & Shasha, 1995); (2) KV keep/evict as online caching with variable sizes and recompute costs, where LRU-style and Landlord policies are 2-competitive with twice the memory (Sleator & Tarjan, 1985; Young, 2002), using the predicted tool return time as the next-use estimate; and (3) a composition proof that charges eviction recompute to the speed augmentation.

Verification. Every algebraic step of Lemma 2 is proved with Z3 (the negation is unsatisfiable over the reals; Z3 also finds a counterexample when the batch condition is dropped). Theorem 1, McNaughton's rule, Lemma 2(i) and Lemma 3 are checked exhaustively on small instances and by randomized property tests, along with a counterexample to Lemma 3 when tools share a slot (issue order then matters); 663 tests pass.

3. System

Runtime. A deadline-enforcing agent runtime in front of vLLM. Each task is an agent program (a single agent that fans out parallel searches, or an orchestrator with worker agents); tool calls are issued asynchronously as soon as their inputs exist (Lemma 3), and one timeout covers planning, tools, every model call and the answer, cancelling outstanding work at the deadline. Every event is traced, and from the traces we compute serving metrics (GPU busy and idle time during tool waits, prefill tokens computed vs. reused).

Serving layer. Many tasks share one vLLM server. For each model request the layer sets the engine priority (FCFS; EDF by task deadline; or least laxity, i.e. the task deadline minus the estimated remaining critical path) through vLLM's priority scheduling, and with admission control it drops a task whose estimated finish, stretched by the current batch slowdown, is past its deadline. The same code runs against a simulated continuous-batching engine (step time $a + cB$ plus prefill, KV capacity, priority queue) for CPU experiments.

Cost model. 48 profiled requests on the serving stack fit an affine model: 0.18 ms per prefill token, 29.3 ms per decode token ($R^2 = 0.9999$, median error 0.4%). Decoding with B concurrent sequences ($B = 1, \ldots, 128$) fits $\tau(B) = a + cB$ with $a = 27.8$ ms and $c = 0.543$ ms ($R^2 = 0.9992$, at most 3% error per point), so Lemma 2's $m = a/c \approx 51$: batching up to $\sim$51 requests at most doubles each request's per-token time.

KV preloading. The final prompt is an append-only log of tool results, so each *preload* request (the log so far, one output token) is a prefix of the final prompt and vLLM's automatic prefix caching reuses its KV blocks. Preloads are charged to the deadline and issued only while tools are still running; the cache is reset before every task so runs are independent.

4. Initial evaluation

Setup. Qwen3-8B (Qwen Team, 2025) on vLLM 0.32.0, one A40, one task at a time (the first slice of setting a). Tasks are held-out HotpotQA questions (Yang et al., 2018). The agent plans 8 searches and issues them asynchronously at once over a pooled corpus (the question's 10 paragraphs plus 90 distractor paragraphs), top-10 passages each, $\sim$1.4k

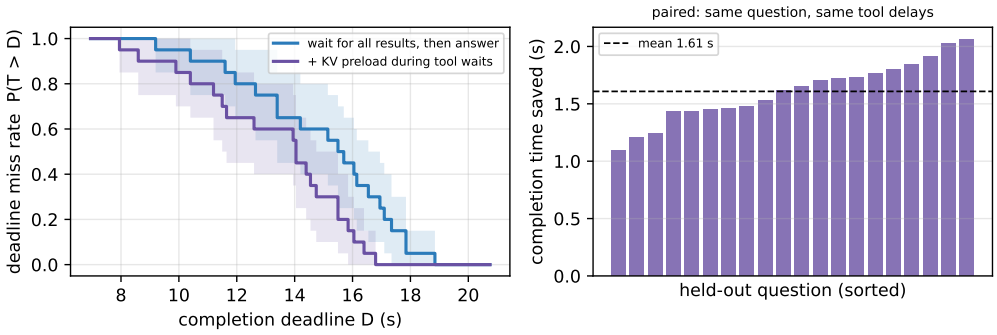


Figure 1. **Asynchronous tool calls with KV preloading on vLLM** (20 held-out questions, 8 parallel searches per question). Left: deadline miss rate $P(T > D)$ vs. deadline, 95% bootstrap bands over questions. Right: paired completion-time saving of preloading per question (same question, same tool delays).

Table 1. Serving view (means over the 20 questions). GPU util. = fraction of the tool-wait window in which the model is busy; cached = share of the final request's prompt served from the prefix cache.

policy	end (s)	GPU util. during tools	model s after last return	GPU s total	cached (final)
wait for all, then answer	14.86	0.00	2.34	6.1	0%
+ KV preload	13.26	0.28	0.73	6.6	87%

tokens per result and an ∼11k-token final prompt; no context cap. Tool delays are reproducible draws (Uniform[0.5,4] s, or with probability 0.3 Uniform[4,12] s), identical across policies. The policies compared never read the clock, so one run per question gives its completion time T and the miss rate at *every* deadline D is exactly $P(T > D)$ on the same runs.

Result. Preloading finished earlier on 20 of 20 questions, by 1.61 s on average (95% CI 1.49–1.72; range 1.10–2.06), with identical answers on all 20 (Figure 1). At $D = 16$ s the miss rate fell from 0.45 to 0.15. Table 1 shows why: without preloading the GPU is idle for the whole tool wait and the final request prefills ∼11k tokens after the last tool returns; with preloading, 87% of the final prompt comes from the cache and the model time after the last return drops from 2.34 s to 0.73 s. Preloading is not free: 6.6 preload requests per task (median 0.32 s each) used 2.2 s of GPU time to save 1.6 s, which pays off only because that time was otherwise idle. In a multi-tenant setting it competes with other agents, which is exactly the trade-off the general algorithm must make.

Microbenchmark. Isolating the final request (30 measurements), its latency without preloading grows with evidence size (0.79 s at 2.8k tokens, 2.29 s at 10.8k) and stays at 0.39–0.51 s with preloading, with 72–94% of the prompt cached for 4–16 results (server-reported `cached_tokens`). At small evidence sizes the saving is small (0.15–0.3 s at 2–3k tokens).

Multi-tenant simulation (not GPU measurements). On the simulated engine with the measured c ($c = 0.5$ ms; 3 seeds, 120 s of Poisson arrivals, a mix of fan-out and orchestrator tasks, deadlines 1.5–4× the unloaded completion time), FCFS and EDF finish 96–98% of tasks on time up to 1.5 tasks/s but fall to 0.57 at 3 tasks/s and 0.29 at 4 tasks/s. EDF alone does not help, because with up to 256 concurrent sequences requests rarely wait in the engine queue. Admission control raises the on-time fraction to 0.73 and 0.55 by dropping tasks that can no longer finish (21% and 34% of tasks). Running the same layer on the GPU is the next step.

5. Remaining challenges and plan

Theory. Prove the remaining steps of Section 2: the sub-deadline assignment and its loss, the KV caching reduction with recompute costs, and the composed constant-factor guarantee; extend Lemma 2 to steps that mix prefill and decode. **System.** Run the multi-tenant layer on the GPU; implement the policy as a vLLM layer (request priorities and admission first, a scheduler change only if needed), including KV keep/evict decisions during tool waits; add collaborating-agent tasks (orchestrator and sub-agents) and the synchronous vs. asynchronous choice. **Evaluation.** Real tools with real latencies (search APIs, code execution) and estimated latency distributions; the model's own context limit; multi-tenant arrivals at increasing load; baselines: stock vLLM scheduling with synchronous tool calls, asynchronous calls without our scheduler. Metrics: on-time completion rate vs. load, tail latency, throughput, GPU and KV utilization. **Limitations of current results:** one task at a time on one GPU and one model; 20 questions; injected delays with a known distribution; preloading gains depend on evidence size.

References

Abhyankar, R., He, Z., Srivatsa, V., Zhang, H., and Zhang, Y. INFERCEPT: Efficient intercept support for augmented large language model inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Baruah, S., Koren, G., Mao, D., Mishra, B., Raghunathan, A., Rosier, L., Shasha, D., and Wang, F. On the competitiveness of on-line real-time task scheduling. *Real-Time Systems*, 4(2):125–144, 1992.

Kalyanasundaram, B. and Pruhs, K. Speed is as powerful as clairvoyance. *Journal of the ACM*, 47(4):617–643, 2000.

Karp, R. M. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pp. 85–103. Plenum Press, 1972.

Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M. W., Keutzer, K., and Gholami, A. An LLM compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Koren, G. and Shasha, D. D^{over}: An optimal on-line scheduling algorithm for overloaded uniprocessor real-time systems. *SIAM Journal on Computing*, 24(2):318–339, 1995.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Lenstra, J. K., Rinnooy Kan, A. H. G., and Brucker, P. Complexity of machine scheduling problems. *Annals of Discrete Mathematics*, 1:343–362, 1977.

Lin, C., Han, Z., Zhang, C., Yang, Y., Yang, F., Chen, C., and Qiu, L. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

McNaughton, R. Scheduling with deadlines and loss functions. *Management Science*, 6(1):1–12, 1959.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Sleator, D. D. and Tarjan, R. E. Amortized efficiency of list update and paging rules. *Communications of the ACM*, 28(2):202–208, 1985.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

Young, N. E. On-line file caching. *Algorithmica*, 33(3): 371–383, 2002.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.